OTEL-03: Collector Deployment Patterns

> **Series:** OTEL — OpenTelemetry Integration | **Notebook:** 3 of 8 | **Created:** January 2026 | **Last Updated:** 05/19/2026

Deploying the OpenTelemetry Collector

The OTel Collector can be deployed in various patterns depending on your infrastructure. This notebook covers deployment modes, Kubernetes configurations, and best practices for production.

Table of Contents

Prerequisites

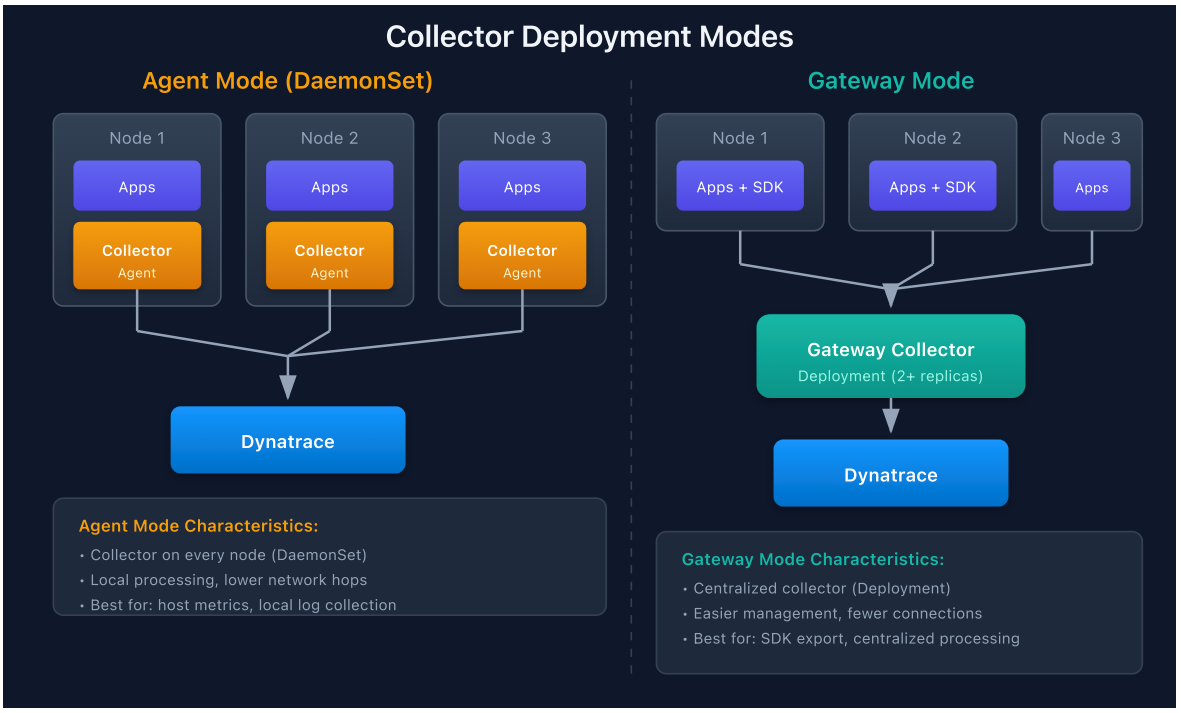
Requirement	Details
Knowledge	OTEL-02: Collector Architecture
Tools	kubectl, Helm, or Docker

1. Deployment Modes

Mode Comparison

Mode	Deployment	Use Case	Pros	Cons
Agent	DaemonSet/Sidecar	Per-node collection	Low latency, local processing	More instances
Gateway	Deployment	Centralized	Single point, aggregation	Potential bottleneck
Hybrid	Both	Large scale	Best of both	Complex

Architecture Patterns



2. Agent Mode (Sidecar/DaemonSet)

DaemonSet Configuration

> **Important:** Always pin your Collector image to a specific version. Using `latest` can cause unexpected behavior during upgrades. Check the OpenTelemetry Collector releases for the current stable version.

```yaml
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: otel-collector-agent
  namespace: otel
spec:
  selector:
    matchLabels:
      app: otel-collector-agent
  template:
    metadata:
      labels:
        app: otel-collector-agent
    spec:
      containers:
        - name: collector
          image: otel/opentelemetry-collector-contrib:0.120.0
          args:
            - --config=/conf/otel-collector-config.yaml
          ports:
            - containerPort: 4317  # OTLP gRPC
            - containerPort: 4318  # OTLP HTTP
          resources:
            requests:
              cpu: 100m
              memory: 256Mi
            limits:
              cpu: 500m
              memory: 512Mi
          volumeMounts:
            - name: config
              mountPath: /conf
      volumes:
        - name: config
          configMap:
            name: otel-collector-config
```

Sidecar Pattern

```
apiVersion: v1
kind: Pod
metadata:
  name: my-app
spec:
  containers:
    - name: app
      image: my-app:latest
      env:
        - name: OTEL_EXPORTER_OTLP_ENDPOINT
          value: http://localhost:4317
    - name: otel-collector
      image: otel/opentelemetry-collector-contrib:0.120.0
      args:
        - --config=/conf/otel-collector-config.yaml
      ports:
        - containerPort: 4317
```

3. Gateway Mode

Gateway Deployment

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: otel-collector-gateway
  namespace: otel
spec:
  replicas: 2  # HA
  selector:
    matchLabels:
      app: otel-collector-gateway
  template:
    metadata:
      labels:
        app: otel-collector-gateway
    spec:
      containers:
        - name: collector
          image: otel/opentelemetry-collector-contrib:latest
          args:
            - --config=/conf/otel-collector-config.yaml
          ports:
            - containerPort: 4317
            - containerPort: 4318
          resources:
            requests:
              cpu: 500m
              memory: 1Gi
            limits:
              cpu: 2000m
              memory: 4Gi
```

Gateway Service

```
apiVersion: v1
kind: Service
metadata:
  name: otel-collector-gateway
  namespace: otel
spec:
  ports:
    - name: otlp-grpc
      port: 4317
      targetPort: 4317
    - name: otlp-http
      port: 4318
      targetPort: 4318
  selector:
    app: otel-collector-gateway
```

4. Kubernetes Deployment

Helm Installation

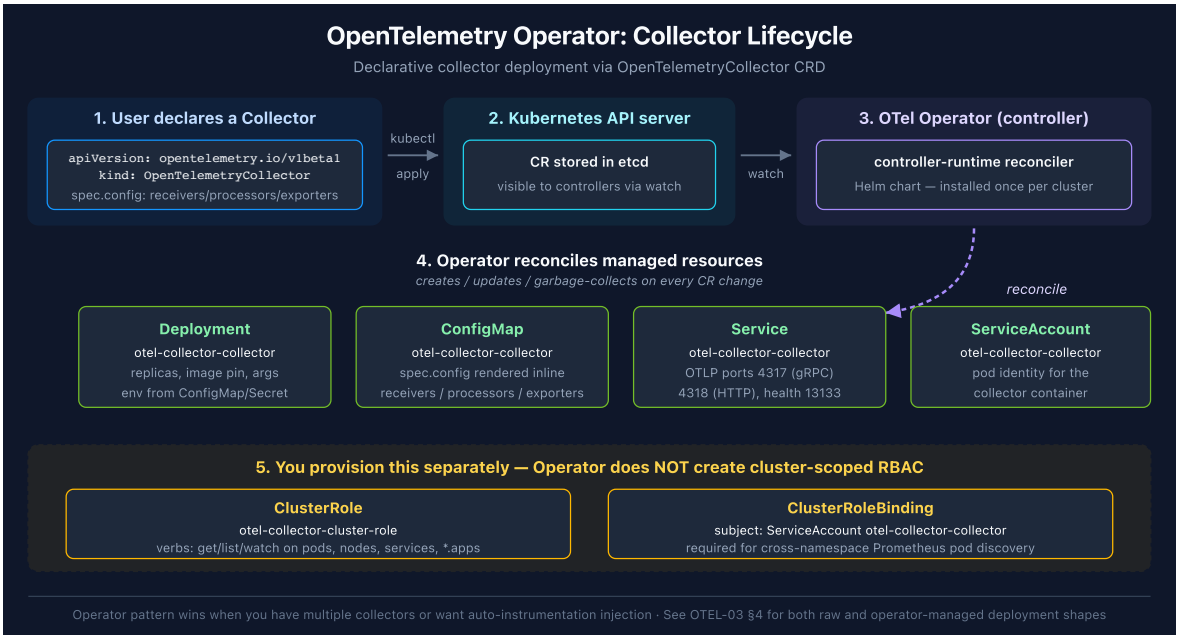
```
# Add Helm repo
helm repo add open-telemetry https://open-telemetry.github.io/opentelemetry-
helm-charts
helm repo update

# Install as DaemonSet (agent mode)
helm install otel-collector open-telemetry/opentelemetry-collector \
  --namespace otel \
  --create-namespace \
  --set mode=daemonset

# Install as Deployment (gateway mode)
helm install otel-collector open-telemetry/opentelemetry-collector \
  --namespace otel \
  --create-namespace \
  --set mode=deployment \
  --set replicaCount=2
```

Helm Values for Dynatrace

```
# values.yaml
mode: deployment
replicaCount: 2

config:
  receivers:
    otlp:
      protocols:
        grpc:
          endpoint: 0.0.0.0:4317
        http:
          endpoint: 0.0.0.0:4318

  processors:
    batch:
      timeout: 10s
    memory_limiter:
      check_interval: 1s
      limit_mib: 800

  exporters:
    otlphttp:
      endpoint: https://${DT_ENDPOINT}/api/v2/otlp
      headers:
        Authorization: Api-Token ${DT_TOKEN}

  service:
    pipelines:
      traces:
        receivers: [otlp]
        processors: [memory_limiter, batch]
        exporters: [otlphttp]
      metrics:
        receivers: [otlp]
        processors: [memory_limiter, batch]
        exporters: [otlphttp]
      logs:
        receivers: [otlp]
        processors: [memory_limiter, batch]
        exporters: [otlphttp]
```

OpenTelemetry Operator (CRD-managed)

The Helm install above creates a raw `Deployment` that you own and roll over manually. The **OpenTelemetry Operator** introduces a higher-level pattern — install the operator once, then declare collectors as `OpenTelemetryCollector` custom resources. The operator reconciles the `Deployment`, `ConfigMap`, `ServiceAccount`, and the collector's container.



Two-stage install:

```
# Stage 1 — install the operator (once per cluster)
helm repo add open-telemetry https://open-telemetry.github.io/opentelemetry-
helm-charts
helm repo update

helm install opentelemetry-operator open-telemetry/opentelemetry-operator \
  --namespace opentelemetry-operator-system \
  --create-namespace \
  --set "manager.collectorImage.repository=otel/opentelemetry-collector-k8s"
\
  --set admissionWebhooks.certManager.enabled=false \
  --set admissionWebhooks.autoGenerateCert.enabled=true \
  --wait --timeout 5m

# Stage 2 — declare a collector via custom resource
kubectl apply -f opentelemetry-collector.yaml
```

`OpenTelemetryCollector` resource shape:

```yaml
apiVersion: opentelemetry.io/v1beta1
kind: OpenTelemetryCollector
metadata:
  name: otel-collector
  namespace: o11y
spec:
  env:
    - name: DT_ENDPOINT
      valueFrom:
        configMapKeyRef:
          name: otel-collector-config
          key: DT_ENDPOINT
    - name: DT_API_TOKEN
      valueFrom:
        secretKeyRef:
          name: otel-collector-secret
          key: DT_API_TOKEN
  config:
    receivers:
      prometheus:
        config:
          scrape_configs:
            - job_name: kubernetes-pods
              # See OTEL-05 §6 for the full receiver + relabel chain
    processors:
      memory_limiter:
        check_interval: 1s
        limit_percentage: 75
      batch:
        send_batch_size: 10000
        timeout: 10s
    exporters:
      otlphttp:
        endpoint: ${env:DT_ENDPOINT}
        headers:
          Authorization: Api-Token ${env:DT_API_TOKEN}
    service:
      pipelines:
        metrics:
          receivers: [prometheus]
          processors: [memory_limiter, batch]
          exporters: [otlphttp]
```

The operator creates the `ServiceAccount` automatically (named `<collectorName>-collector`). For Prometheus pod discovery the collector still needs cluster-scoped read access — provision a `ClusterRole` and bind it separately:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: otel-collector-cluster-role
rules:
  - apiGroups: ["", "apps", "batch"]
    resources:
      - pods
      - namespaces
      - nodes
      - services
      - replicasets
      - deployments
      - daemonsets
      - statefulsets
      - jobs
      - cronjobs
    verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: otel-collector-cluster-role-binding
subjects:
  - kind: ServiceAccount
    name: otel-collector-collector   # &lt;collectorName&gt;-collector
    namespace: o11y
roleRef:
  kind: ClusterRole
  name: otel-collector-cluster-role
  apiGroup: rbac.authorization.k8s.io
```

When to use which:

Choose	When
Raw `Deployment` + Helm chart	One or two collectors; team already manages Deployment rollouts; Helm-first workflows
`OpenTelemetryCollector` CRD + Operator	Multiple collectors per cluster; auto-reload on config change; want OTLP auto-instrumentation injection for application pods

The operator's auto-instrumentation feature is the strongest pull for greenfield Kubernetes deployments — annotate a pod with `instrumentation.opentelemetry.io/inject-python: "true"` and the operator injects an

init container that loads the OTel agent, pointed at a `Service` your collector exposes. See **OTEL-04 — Trace Instrumentation** for the application-side pattern.

> **Sources:**
> - OpenTelemetry Operator (opentelemetry-operator GitHub)
> - OpenTelemetryCollector CRD reference (opentelemetry-operator GitHub)
> - **Derived:** "when to use which" table combines operator design with operational trade-offs observed in community deployments

5. Docker Compose

Basic Setup

```
# docker-compose.yaml
version: '3.8'

services:
  otel-collector:
    image: otel/opentelemetry-collector-contrib:latest
    command: ["--config=/etc/otel-collector-config.yaml"]
    volumes:
      - ./otel-collector-config.yaml:/etc/otel-collector-config.yaml
    ports:
      - "4317:4317"   # OTLP gRPC
      - "4318:4318"   # OTLP HTTP
      - "13133:13133" # Health check
    environment:
      - DT_ENDPOINT=${DT_ENDPOINT}
      - DT_TOKEN=${DT_TOKEN}

  my-app:
    image: my-app:latest
    environment:
      - OTEL_EXPORTER_OTLP_ENDPOINT=http://otel-collector:4317
      - OTEL_SERVICE_NAME=my-app
    depends_on:
      - otel-collector
```

6. High Availability

HA Considerations

Component	HA Strategy
Agent (DaemonSet)	Node failure = pod restarts
Gateway	Multiple replicas behind load balancer
Persistent Queue	Prevent data loss during restarts

Gateway with Persistent Queue

```
exporters:
  otlphttp:
    endpoint: https://${DT_ENDPOINT}/api/v2/otlp
    sending_queue:
      enabled: true
      num_consumers: 10
      queue_size: 1000
      storage: file_storage
    retry_on_failure:
      enabled: true
      initial_interval: 1s
      max_interval: 30s
      max_elapsed_time: 300s

extensions:
  file_storage:
    directory: /var/lib/otel/storage
    timeout: 10s

service:
  extensions: [file_storage]
```

Pod Disruption Budget

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: otel-collector-pdb
spec:
  minAvailable: 1
  selector:
    matchLabels:
      app: otel-collector-gateway
```

7. Resource Sizing

Sizing Guidelines

Throughput	CPU	Memory	Replicas
Low (<1000 spans/s)	100m	256Mi	1
Medium (<10k spans/s)	500m	1Gi	2
High (<100k spans/s)	2	4Gi	3+
Very High (>100k spans/s)	4+	8Gi+	5+

Memory Limiter Tuning

```
processors:
  memory_limiter:
    check_interval: 1s
    limit_mib: 800       # 80% of container memory limit
    spike_limit_mib: 200 # Room for spikes
```

Horizontal Pod Autoscaler

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: otel-collector-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: otel-collector-gateway
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

8. Security Considerations

Token Management

Store Dynatrace API tokens in Kubernetes Secrets:

```
# Kubernetes Secret (create via kubectl or sealed-secrets)
apiVersion: v1
kind: Secret
metadata:
  name: dynatrace-otel-token
type: Opaque
stringData:
  token: &lt;your-dynatrace-api-token&gt;
```

```
# Reference in Deployment
env:
  - name: DT_TOKEN
    valueFrom:
      secretKeyRef:
        name: dynatrace-otel-token
        key: token
```

TLS Configuration

```
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
        tls:
          cert_file: /certs/server.crt
          key_file: /certs/server.key
          client_ca_file: /certs/ca.crt  # mTLS
```

Network Policy

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: otel-collector-policy
spec:
  podSelector:
    matchLabels:
      app: otel-collector-gateway
  policyTypes:
    - Ingress
    - Egress
  ingress:
    - from:
        - namespaceSelector: {}
      ports:
        - protocol: TCP
          port: 4317
        - protocol: TCP
          port: 4318
  egress:
    - to:
        - ipBlock:
            cidr: 0.0.0.0/0
      ports:
        - protocol: TCP
          port: 443
```

Summary

In this notebook, you learned:

- Deployment modes: Agent vs. Gateway vs. Hybrid
- DaemonSet and Sidecar configurations for agent mode
- Gateway deployment with services
- Kubernetes deployment via Helm
- Docker Compose setup
- High availability with persistent queues
- Resource sizing guidelines
- Security best practices

Next Steps

Next Notebook	Topic
OTEL-04: Trace Instrumentation	Instrumenting code
OTEL-07: Dynatrace Integration	Complete DT setup

References

- Collector Deployment
- Helm Chart
- Docker Images